EXPERIMENT NO: 6

Student Name: Payal Singh
UID: 23BCS13626
Branch: BE-CSE
Section/Group: KRG-1B
Semester: 6th
Date of Performance: 28/02/2026
Subject Name: System Design
Subject Code: 23CSH-314

Aim

To design a highly scalable video streaming platform similar to YouTube that supports heavy video uploading, adaptive bitrate streaming, and real-time social interactions with high availability and low latency.

Objectives

- Understand the architecture of a massive-scale media streaming system.
- Identify functional requirements such as direct-to-cloud uploads and personalized feeds.
- Identify non-functional requirements including CDN caching, high throughput, and latency constraints.
- Analyze the decoupling of synchronous API requests using event-driven stream processing.
- Design REST APIs for video management, search, and user interactions.

Procedure

1. Studied real-world video streaming architectures and content delivery networks.
2. Identified core entities such as Users, Videos, Interactions, and View Histories.
3. Analyzed the Pre-Signed URL pattern to bypass API Gateway bottlenecks for large file uploads.
4. Collected functional and non-functional requirements for media delivery.
5. Designed APIs for video processing, metadata retrieval, and social engagement.
6. Evaluated streaming protocols (HLS/DASH) and separated thumbnail storage for optimization.
7. Analyzed stream processing (Apache Flink/Kafka) for real-time, high-volume view aggregation.

Functional Requirements

YOUTUBE SYSTEM DESIGN

Functional Requirements :-

① Video Streaming.
- ↳ Watch videos
- ↳ Adaptive bitrate streaming → (HLS/DASH)
  (144p, 240p, 360p, 480p, 720p, 1080p, 4k)
  depending on bandwidth
- ↳ Auto adjust quality based on b/w
- ↳ Resume playback

② User Interaction
- ↳ Like / Dislike
- ↳ Comment
- ↳ Share
- ↳ Save to playlist
- ↳ Watch history

③ Subscriptions
- ↳ Subscribe / Unsubscribe
- ↳ Notifications from subscribed channels → PUB-SUB Model
- ↳ Observer Pattern (Channel → Subscribers)
- ↳ Tiers → Free, Basic, Premium, Enterprise → Strategy Pattern

④ Search (Elastic Search by AWS)
- ↳ Title, tag, description / channel
- ↳ Auto-recommendation
- ↳ Filters - When was it uploaded (week, month, year), views, duration, type

* Redis Cache - Trending, Popular videos.

1.

(5) Notifications

(6) Content Creation → Upload video (AWS S3)
→ Live Stream
→ Create Post
→ Create Playlist

Note:- Hr video ka bhi to metadata hoga ek.
- id - title - description - tags - upload time
- views count - like count - duration etc.

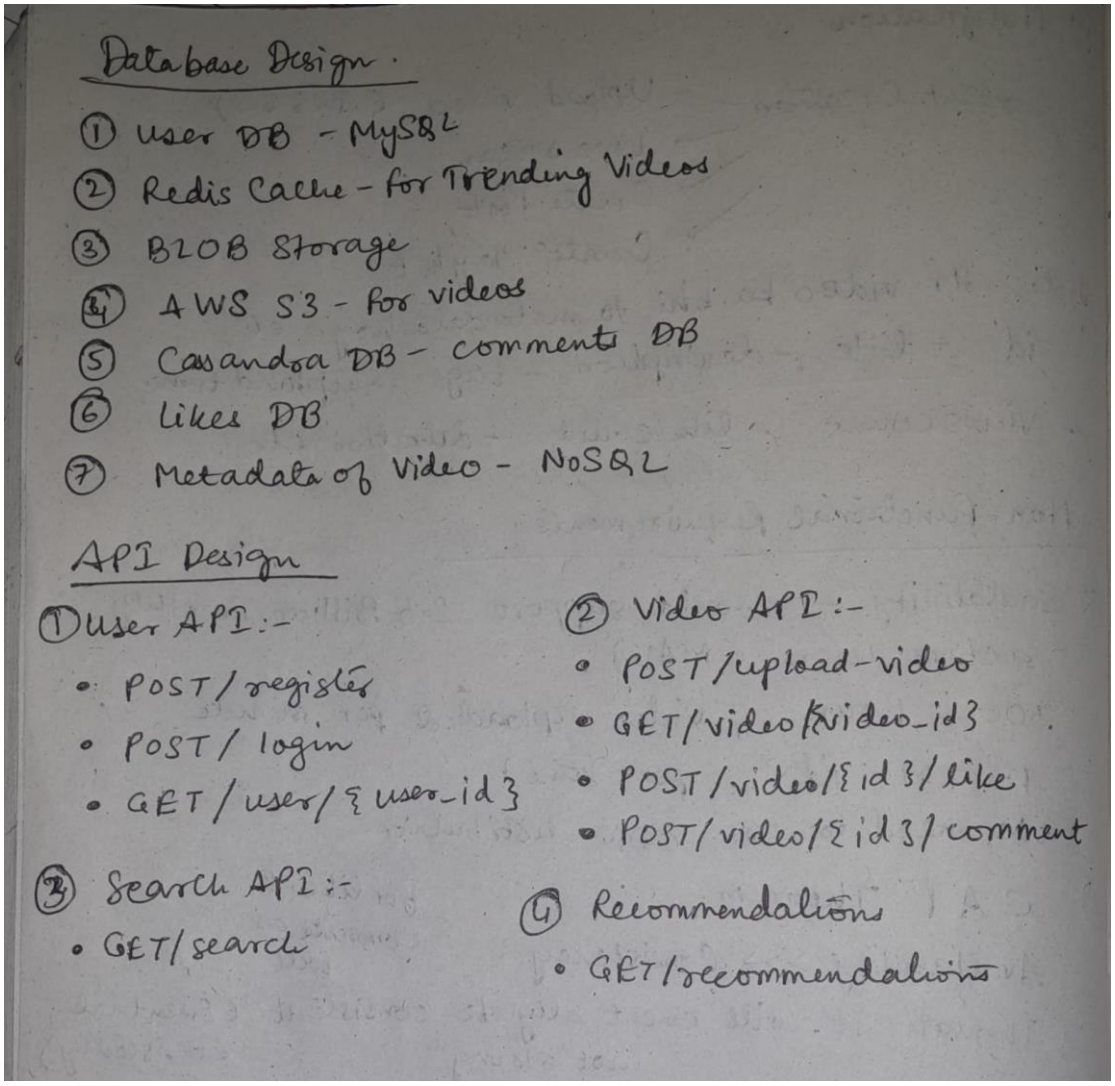
Non-Functional Requirements.

(1) Scalability - should support 2.5 Billion monthly active users (MAU)
500+ hours of video uploaded per minute
1-10 M RPS (peak time)
∴ we need to use CDN distribution

(2) CAP Theorem.
Availability >>> Consistency.
Though it will eventually be consistent (Eventual Consistency) not strong
for likes & comments as well.
& Partition tolerance

(3) Latency = Network + Database + Application
(i) Video start time should be fast (1000-2000 ms)
(ii) Search response should be fast (100-300 ms)

(4) Security.
Authentication, Rate limiting, Copyright detection (AI),
Content moderation
↳ Moderation + kafka
as soon as video uploaded.



Database Design.

1. User DB - MySQL
2. Redis Cache - for Trending Videos
3. BLOB Storage
4. AWS S3 - for videos
5. Cassandra DB - comments DB
6. Likes DB
7. Metadata of Video - NoSQL

API Design

1. User API:-
   - POST/register
   - POST/login
   - GET/user/{user_id}
2. Video API:-
   - POST/upload-video
   - GET/video/{video_id}
   - POST/video/{id}/like
   - POST/video/{id}/comment
3. Search API:-
   - GET/search
4. Recommendations
   - GET/recommendations

Core Entities of the System

- Video Streaming
- User Interaction
- Subscriptions
- Search
- Notifications
- Content Creation

4. API Endpoints

A. User & Channel APIs

1. User Registration: POST /api/v1/users/register
2. User Login: POST /api/v1/users/login
3. Channel Profile Display: GET /api/v1/channels/{channel_id}
4. Channel Profile Update: PUT /api/v1/channels/{channel_id}

B. Video Management APIs

1. Upload Video: POST /api/v1/videos/upload
2. Get Video Details: GET /api/v1/videos/{video_id}
3. Update Video Metadata: PUT /api/v1/videos/{video_id}
4. Delete Video: DELETE /api/v1/videos/{video_id}
5. Fetch Home Feed: GET /api/v1/videos/feed?limit={limit}&offset={offset}
6. Fetch Subscription Feed: GET /api/v1/users/{user_id}/subscriptions/feed

C. User Interactions

1. Like Video: POST /api/v1/videos/{video_id}/like
2. Remove Like: DELETE /api/v1/videos/{video_id}/unlike
3. Post Comment: POST /api/v1/videos/{video_id}/comments
4. Fetch Comments: GET /api/v1/videos/{video_id}/comments
5. Subscribe to Channel: POST /api/v1/channels/{channel_id}/subscribe
6. Unsubscribe: DELETE /api/v1/channels/{channel_id}/unsubscribe

Non-Functional Requirements

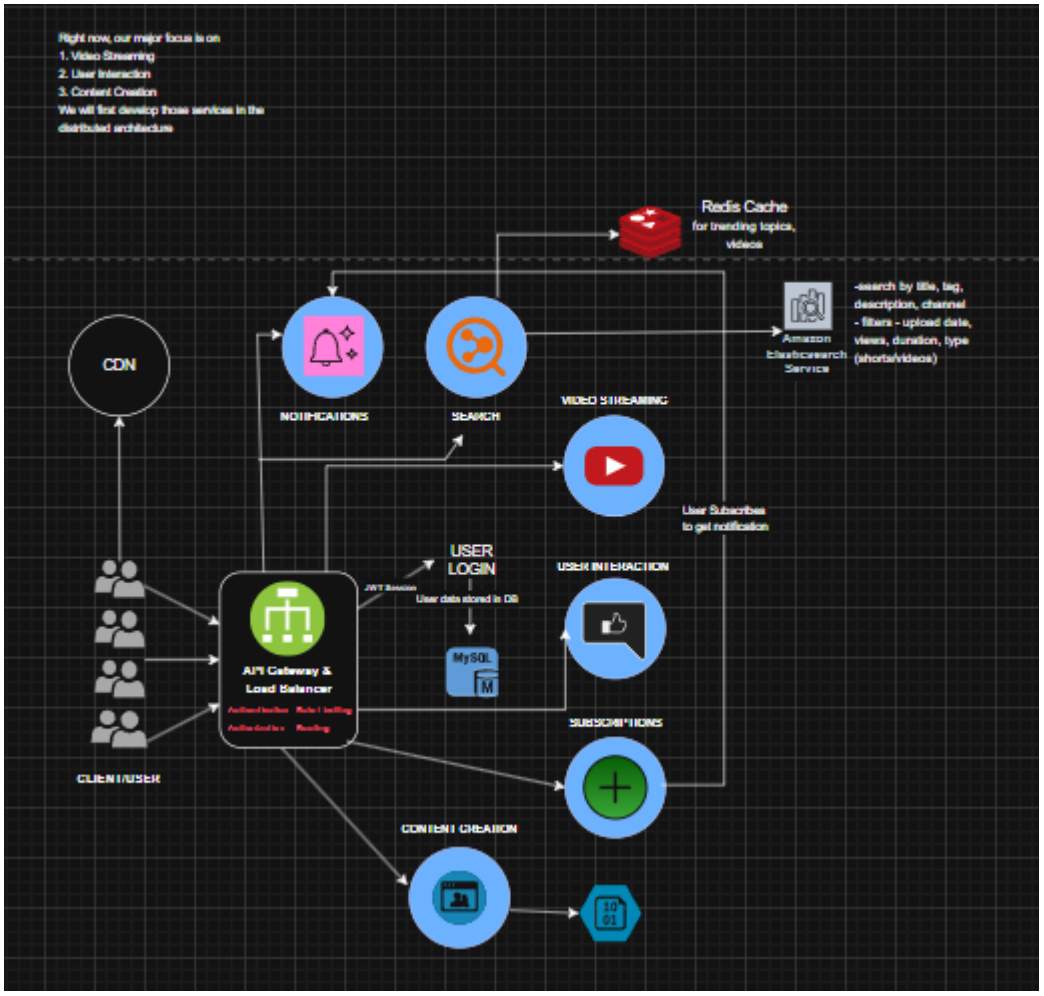
1. System must handle extremely high throughput for concurrent uploads and millions of simultaneous views.
2. High Availability must be prioritized, ensuring no single point of failure.
3. System should follow an Eventual Consistency model for social metrics (view counts, likes) to optimize database write loads.
4. Video delivery latency should be minimized via Edge Caching and a Global CDN.
5. Transcoding pipeline must be horizontally scalable to handle viral upload spikes.

High Level Design (HLD)

The system consists of Client Applications, an API Gateway, and specialized microservices (Auth, Upload, Video Metadata, Feed, and Interactions). Heavy uploads bypass the backend via Pre-Signed URLs directly to Object Storage. Asynchronous transcoding pipelines process videos into HLS/DASH segments, which are delivered globally via a CDN. High-volume interactions are managed using Redis (Likes), NoSQL clusters (Comments), and a Kafka-to-Apache Flink stream processing pipeline for accurate view aggregation.

Outcome

- Designed a highly scalable, event-driven video streaming architecture.
- Identified critical functional and non-functional requirements for media delivery.
- Designed REST APIs and decoupled asynchronous processing pipelines.
- Understood the application of CDNs, adaptive bitrate streaming, and stream processing for massive-scale systems.


Right now, our major focus is on
1. Video Streaming
2. User Interaction
3. Content Creation
We will first develop those services in the distributed architecture
Redis Cache
for trending topics, videos
CDN
NOTIFICATIONS
SEARCH
Amazon Elasticsearch Service
-search by title, tag, description, channel
- filters - upload date, views, duration, type (shorts/videos)
VIDEO STREAMING
User Subscribes to get notification
USER LOGIN
JWT Session
User data stored in DB
MySQL
USER INTERACTION
API Gateway & Load Balancer
SUBSCRIPTIONS
CLIENT/USER
CONTENT CREATION